

כתבו למחלקה WeatherService בדיקות יחידה. אין צורך לזכור תחביר מדויק:

```
public class WeatherServiceTest {
    private WeatherService weatherService;
    private WebService webServiceMock;

    @BeforeEach
    public void setUp() {
        webServiceMock = Mockito.mock(WebService.class);
        this.weatherService = new WeatherService(webServiceMock);
    }

    @ParameterizedTest
    @CsvSource({"25.0", "0.0", "-10.0", "10.0", "5.3", "150.0"})
    public void getTemperature_celsius_success(double temp) {
        when(webServiceMock.getTemperature()).thenReturn(temp + "");
        double weather = weatherService.getTemperature(true);
        assertEquals(temp, weather, "Temperature should be " + temp);
    }

    @ParameterizedTest
    @CsvSource({"25.0, 77", "0.0, 32.0", "-10.0, 14.0", "10.0, 50.0",
        "5.3, 41.54", "150.0, 302.0"})
    public void getTemperature_fahrenheit_success(double celsius,
        double fahrenheit) {
        when(webServiceMock.getTemperature()).thenReturn(celsius + "");
        double weather = weatherService.getTemperature(false);
        assertEquals(fahrenheit, weather,
            "Temperature should be " + fahrenheit);
    }
}
```

#### הערות:

1. כל קלט הוא חוקי ולכן אין בדיקות rainy-day. אין סיבה לבדוק מה קורה אם webService מחזיר מחרוזת שהיא לא מספר כי בבדיקות יחידה אנחנו מניחים נכונות של מחלקות אחרות.
2. אין צורך לבדוק את הפונ' private משתי סיבות: (א) הפונ' הזו היא בסה"כ הגדרה; (ב) היא פרט מימוש ובאותה מידה בגרסה אחרת של המערכת אפשר יהיה להכניס אותה לתוך הפונ' getTemperature.
3. יש לכתוב מספר בדיקות של תקינות ולא אחד (בפתרון זה מופיע כמספר ערכים csv). הסיבה היא שאולי הקוד מתנהג בצורה לא תקינה (כרגע/בגרסה עתידית). גם אם טמפרטורה אחת מכסה את כל מסלולים בקוד — מי אמר שהקוד נכון? שהוא עושה את מה שהוא אמור לעשות?
4. אמנם לא צריך לזכור תחביר מדויק אבל יש ציפייה שיופיעו הרעיונות של: beforeAll, שימוש נכון ב when then (להבין למי עושים את זה), של parameterized test. כמו-כן יש להשתמש בקונבציה של שמות הטסטים (כפי שמופיע בפתרון).
5. טעות נפוצה — עטיפה של weatherService עם spy. זו טעות כיוון שאין בזה צורך. הוא לא מאפשר

לנו בדיקה נוספת מעבר למה שצריך.

6. טעות נפוצה - בדיקה של `parseDouble` או `mock של Double`. אין שום סיבה לבדוק את זה - אנחנו מניחים שמחלקות אחרות ממומשות נכון.